

第四章 进程、作业、服务、日志与计划任务

进程处理正在运行的实例，systemd 处理受管理的 unit，日志提供失败证据，计划任务则在另一时间和有限环境中启动命令。把它们混在一起，容易出现“手工执行成功、定时执行失败”或“服务已启动、重启后未运行”。本章按运行实例、管理声明、证据和调度四层组织操作。

[概念] 进程是程序的一次运行实例，具有 PID、父 PID、用户、状态、优先级和打开资源；Shell job 是当前 Shell 对一个或多个进程的交互管理视图；systemd unit 是服务、socket、target、mount 或 timer 等受管理对象的声明。

[概念] service 的 active 状态描述当前运行，enabled 描述启动依赖中是否建立持久链接，masked 表示禁止启动。日志记录是证据，不等于服务终态；时间和时区错误会直接影响日志关联与计划任务。

[操作语义] ps / pgrep / top 查询进程，kill / pkill 发送信号，nice / renice 调整调度优先级；jobs / fg / bg 只管理当前 Shell 的作业表。

[操作语义] systemctl 查询和改变 unit 当前及启动状态；journalctl 从 systemd journal 取证；crontab 描述周期任务，at 提交一次性任务，systemd timer 把调度与 service unit 组合。

[知识专题] 进程、作业、信号与优先级

① [知识点] 先选择稳定标识再操作

PID 唯一标识当前进程实例，但重启后可复用。ps -ef 适合父子关系，ps aux 适合资源概览，ps -eo 可选择稳定字段；pgrep -a 按名称查 PID 和命令行，pidof 适合已知程序名。

```
ps -eo pid,ppid,user,stat,ni,etimes,cmd --sort=-etimes
pgrep -a -u appsvc worker
```

操作前核对用户、完整命令行和父子关系。仅按模糊名称 pkill 可能命中不相关进程。

② [知识点] 信号是请求，不是删除进程

kill <PID> 默认发送 SIGTERM (15)，允许程序清理退出；SIGKILL (9) 由内核立即终止，进程无法处理或清理，应只在 TERM 无效且对象确认后使用。SIGHUP (1) 常被守护进程用于重新加载，但具体语义由程序定义。

```
kill -TERM 1234          # 请求有序退出
ps -p 1234 -o pid,stat,cmd # 验证是否仍存在
kill -KILL 1234          # 最后手段
```

僵尸进程已经退出，只等待父进程回收，向僵尸发送更多信号无效；应调查父进程。不可中断睡眠状态 D 常在等待内核 I/O，SIGKILL 也要等阻塞返回。

③ [知识点] nice 值越大, CPU 优先级越低

普通 nice 范围通常为 -20 到 19, 数值越小优先级越高。普通用户可降低自己进程优先级, 但提高优先级需要相应权限。nice 只影响调度倾向, 不是 CPU 配额或性能完成证明。

④ [边界] Shell job 只存在于所属 Shell

`command &` 后台启动, `jobs` 列出当前 Shell 作业, `fg %1` 拉回前台, `bg %1` 继续已停止作业。关闭终端可能向作业发送 SIGHUP; 需要长期服务时应使用 `systemd` 或合适会话工具, 不把 `nohup` 当成服务管理替代品。

[Cheatsheet] 查实例: `ps -eo / pgrep -a`; 先 TERM 后验证, 必要时 KILL; 僵尸查父进程; nice 越大优先级越低; `jobs/fg/bg` 只属于当前 Shell。

[操作专题] 管理 systemd 当前状态与启动配置

① [知识点] start、enable、mask 作用于不同状态

`systemctl start` 现在启动, `stop` 现在停止; `enable` 建立启动依赖, `disable` 移除; `enable --now` 同时启用并启动。 `mask` 把 unit 链接到 `/dev/null`, 连手工或依赖启动也会被阻止, 解除用 `unmask`。

`restart` 停止后重新启动, 可能中断服务; `reload` 请求进程重新读取配置, 前提是 unit 支持。配置是否支持 reload 应由 `systemctl show`、unit 定义或服务文档确认。

② [操作点] 修改配置后先校验, 再改变进程

```
httpd -t                # 服务专用语法检查
systemctl reload httpd   # 支持时平滑加载
systemctl is-active httpd
systemctl is-enabled httpd
```

unit 文件或 drop-in 变化后执行 `systemctl daemon-reload`, 它让 systemd 重新读取 unit 定义, 不会自动重启服务。应用自身配置变化通常不需要 daemon-reload, 而需要应用校验和 reload/restart。

③ [验证点] status 适合入口, 机器判定用专用查询

`systemctl status` 汇总近期日志和状态, 适合调查; `is-active`、`is-enabled` 适合明确判定。`list-dependencies` 查依赖, `cat` 显示主 unit 和 drop-in, `show -p` 获取属性。

```
systemctl status httpd --no-pager
systemctl cat httpd
systemctl show httpd -p ActiveState,SubState,UnitFileState
ss -lntp | grep ':80'          # 功能链的监听层
```

服务 active 只证明 systemd 认为主进程处于活动状态; 仍需按题意检查监听、日志、外部访问和数据。

[Cheatsheet] 当前: `start/stop/reload/restart`; 持久: `enable/disable`; 禁止启动: `mask`; unit 变化才 `daemon-reload`; 验收 `is-active` + `is-enabled` + 服务功能。

[操作专题] 从 journal 与持久日志提取服务证据

① [参数点] 按 unit、启动轮次、时间和优先级收缩

`journalctl -u <UNIT>` 按 unit, `-b` 当前启动, `-b -1` 上一次启动, `--since/--until` 按时间, `-p err` 按优先级上限, `-f` 跟随新日志。组合过滤比对整份 `journal grep` 更精确。

```
journalctl -u httpd -b --since '-10 min' --no-pager
journalctl -p err..alert -b
journalctl --disk-usage
```

日志时间依赖系统时钟和时区。取证前用 `timedatectl` 确认当前时区与同步状态, 避免把错误时间范围解释为“没有日志”。

② [知识点] journal 持久性取决于存储配置

`journald` 可写入易失 `/run/log/journal` 或持久 `/var/log/journal`。RHEL 配置由 `/etc/systemd/journal.conf` 及 drop-in 控制; 建立持久目录或设置 Storage 后应重启/重载相应服务并验证跨启动记录。不要因为当前能看到日志就断言重启后保留。

`rsyslog` 可把消息按 facility/priority 规则写入文本文件或远端。修改 `/etc/rsyslog.conf` 或 `/etc/rsyslog.d/*.conf` 后使用 `rsyslogd -N1` 检查语法, 再重启服务并用 `logger` 生成受控测试消息。

③ [验证点] 受控事件比等待业务日志更可靠

```
logger -p local0.notice -t exam-check 'logging path test'
journalctl -t exam-check --since '-2 min'
grep -F 'logging path test' /var/log/custom.log
```

测试消息证明当前路由链; 若题目要求持久性, 还需检查配置、目录和启动后记录。日志到达文件不证明轮转策略或远端传输正确。

[Cheatsheet] journal 收缩: `-u` + `-b` + `--since` + `-p`; 时间先看 `timedatectl`; `rsyslog` 配置先 `rsyslogd -N1`; 用 `logger` 产生可识别事件并查两端。

[操作专题] 配置时区、chrony 与时间同步

① [操作点] 时区与同步是独立维度

`timedatectl set-timezone Asia/Shanghai` 改显示时区，不改变 UTC 时间来源；`timedatectl set-ntp true` 请求启用系统配置的网络时间机制。在 RHEL 9 常由 `chronyd` 提供 NTP 客户端服务。

```
timedatectl
chronyc sources -v          # 时间源选择与可达性
chronyc tracking             # 本机同步与偏差状态
```

`chronyc sources` 中 `^*` 常表示当前选中的服务器，`^+` 是可接受候选，`^?` 表示不可达或无有效测量。单看服务 `active` 不能证明已经选到时间源。

② [操作点] 修改时间源后分层验证

编辑 `/etc/chrony.conf` 或 drop-in 时按题意添加 `server/pool`，先检查名称解析和网络，再重启 `chronyd`。大偏差是否立即 `step` 受 `makestep` 配置和运行阶段影响，不应无条件手工改系统时间。

③ [验证点] 服务、源和跟踪三层

```
systemctl is-active chronyd
systemctl is-enabled chronyd
chronyc sources -v
chronyc tracking
```

这组证据分别说明进程、启动配置、可用源/选择和当前时钟跟踪，不由其中一条替代全部。

[Cheatsheet] 时区：`timedatectl set-timezone`；`chronyd` 状态：`systemctl`；源：`chronyc sources -v`；同步质量：`chronyc tracking`；`active` 不等于已同步。

[操作专题] 用 cron 与 at 运行可审计任务

① [知识点] 用户 crontab 与系统 crontab 字段不同

用户 `crontab -e` 每行是五个时间字段加命令：分钟、小时、日、月、星期；`/etc/crontab` 和 `/etc/cron.d/` 在时间字段后多一个执行用户。把两种格式混用会让命令无法按预期执行。

```
15 2 * * * /usr/local/sbin/report.sh >>/var/log/report.log 2>&1
```

日字段和星期字段同时受限时具有特殊 OR 语义，复杂日历需求应现场查 `man 5 crontab`，不要凭直觉组合。

② [操作点] 计划任务必须使用可预测环境

cron 的 PATH、工作目录和环境通常比交互 Shell 精简。使用绝对路径，脚本自身设置必要环境和安全 umask；显式重定向 stdout/stderr。不要依赖 alias、当前目录、交互提示或未导出的变量。

`at <TIME>` 提交一次性任务，`atq` 列表，`at -c <JOB>` 查看展开后的作业，`atrm <JOB>` 删除。提交成功只证明队列中存在，仍要确认 `atd` 服务和执行结果。

③ [验证点] 缩短周期只用于受控验证

先手工以目标用户和最小环境运行脚本，再临时选择较近时间验证，检查输出文件、日志和 `journalctl -u crond / -u atd`。验证后恢复题目周期，避免留下高频任务。

```
crontab -l -u appsvc
systemctl is-active crond
journalctl -u crond --since today
```

④ [边界] systemd timer 是另一套调度对象

timer unit 触发同名或指定 service unit，可用 `systemctl list-timers` 查询，支持相对启动时间和持久补跑等语义。题目明确要求 cron 时不要擅自改用 timer；遇到现有 timer 则同时检查 timer 与被触发 service 的日志。

[Cheatsheet] 用户 cron：5 时间字段 + 命令；系统 cron 多执行用户；任务用绝对路径并重定向；一次性任务 `at/atq/at -c`；验收队列/配置 + 调度服务 + 实际产物。

[诊断专题] 服务或计划任务失败时按运行层取证

① [诊断点] 服务启动失败

先 `systemctl status` 获取主错误，再 `journalctl -u ... -b` 收缩日志；随后运行应用专用语法检查，检查端口冲突、权限和 SELinux。不要反复 restart 冲掉最接近失败时刻的证据。

② [诊断点] 手工成功但 cron 失败

以目标用户在 `env -i` 的精简环境中运行，检查绝对路径、工作目录、权限、变量和错误重定向；再查 `crond` 日志。给 cron 增加完整交互 PATH 可能掩盖脚本依赖，应让脚本明确依赖。

③ [诊断点] enabled 但没有运行

`enabled` 只是启动依赖，当前可能失败、被条件跳过或启动后退出。检查 `is-active`、`status` 和本次启动 `journal`；若 `masked`，先确认为什么被 `mask`，而不是直接 `unmask`。

[Cheatsheet] `status` 定位 → `unit journal` → 应用语法 → 端口/权限/SELinux；cron 差异查用户与环境；`enabled ≠ active`；保留最近失败证据再修改。

[经典任务] 配置服务、持久日志和周期健康检查

服务器已安装 `httpd`，但服务无法启动。修复现有配置而不删除站点内容，使其当前运行并开机启动。将 `local0.notice` 消息持久写入 `/var/log/web-health.log`，每天 02:15 以用户 `monitor` 运行 `/usr/local/sbin/web-health.sh`，追加输出与错误到同一日志。

系统时区应为 `Asia/Shanghai`，`chronyd` 使用题目指定时间源并处于有效跟踪状态。不得通过清空配置、禁用 SELinux 或高频 `cron` 掩盖问题。

验收要求：服务语法、`active/enabled`、监听与本机访问；日志路由和受控消息；`cron` 字段、执行身份、环境与实际产物；时区、`chronyd` 服务、源和 `tracking` 分别有证据。

请先独立完成。参考解答从下一页开始。

[参考解答] 从日志证据修复服务并验证调度链

① [诊断点] 收集服务失败证据

```
systemctl status httpd --no-pager
journalctl -u httpd -b --since '-15 min' --no-pager
httpd -t
ss -lntp
```

根据具体语法或端口证据做最小修正，再运行 `httpd -t`，执行 `systemctl enable --now httpd`，核对 active/enabled、监听和 `curl -I http://localhost/`。

② [操作点] 配置日志路由与时间

在 `/etc/rsyslog.d/web-health.conf` 写入 `local0.notice /var/log/web-health.log` 对应规则，执行 `rsyslogd -N1` 后重启 rsyslog。用唯一消息验证 journal 与文件。

```
timedatectl set-timezone Asia/Shanghai
systemctl enable --now chronyd
chronyc sources -v
chronyc tracking
logger -p local0.notice -t web-health 'route verification'
grep -F 'route verification' /var/log/web-health.log
```

③ [操作点] 配置并验证 cron

先以 monitor 手工运行脚本并修正绝对路径/权限，再写入用户 crontab：

```
15 2 * * * /usr/local/sbin/web-health.sh >>/var/log/web-health.log 2>&1
```

确认 monitor 对日志目标具有题目允许的写入路径；更稳妥的设计可以让脚本使用 `logger`，但应服从现有题意。检查 `crontab -l -u monitor`、crond 状态和受控近时执行产物，最后恢复 02:15。

④ [验证点] 最终分层验收

```
systemctl is-active httpd rsyslog crond chronyd
systemctl is-enabled httpd chronyd
ss -lntp | grep ':80'
curl -I http://localhost/
rsyslogd -N1
crontab -l -u monitor
timedatectl; chronyc sources -v; chronyc tracking
```

[Cheatsheet] status/journal/语法查服务 → active/enabled/监听/访问 → rsyslog 语法 + logger 路由 → 目标用户与精简环境测脚本 → cron 配置 + 实际产物 → 时间四层验收。

[本章收束] 运行实例、管理状态和调度环境各有证据

进程工具观察或影响当前实例，systemd 同时管理当前 unit 与启动依赖，journal/rsyslog 保存不同持久范围的证据，cron/at 在独立时间和环境中启动命令。一个层次成功不会自动推进其他层次。

稳定排错从最近失败日志出发，先做应用语法检查，再改变进程。计划任务则先以目标身份和精简环境验证，再检查调度服务和真实产物。最终把 active、enabled、监听、功能、日志、时间和调度逐项验收。